

Modul Ajar Logika Pemrograman

TMS0633



universitas
MALIKUSSALEH

Asnawi, S.T., M.Sc.

Ahmad Nayan, S.T., M.T.

Ir. Shaki Saptiadi Putra, S.T., M.Eng.

Agustus 2025

Prakata

Puji syukur ke hadirat Allah Subhanahu wa Ta'ala atas segala rahmat dan karunia-Nya, sehingga modul ajar dengan judul “*Modul Ajar Logika Pemrograman – TMS0633*” ini dapat tersusun dan disajikan kepada mahasiswa. Modul ini disusun untuk mendukung proses pembelajaran mata kuliah Logika Pemrograman yang menjadi salah satu fondasi penting dalam pendidikan teknik, khususnya di bidang Teknik Mesin.

Tujuan utama dari modul ini adalah untuk memberikan pemahaman mendasar mengenai logika pemrograman serta penerapannya dalam penyelesaian masalah keteknikan. Penekanan pembelajaran tidak diarahkan pada penguasaan bahasa pemrograman atau perangkat lunak tertentu, melainkan pada pembentukan pola pikir logis dan sistematis dalam merumuskan solusi atas permasalahan yang dihadapi, baik dalam ranah teknik maupun dalam kehidupan sehari-hari. Dengan demikian, mahasiswa diharapkan mampu mengembangkan keterampilan analitis serta mengasah kemampuan dalam menyusun algoritma yang efektif.

Dalam praktik pembelajaran, modul ini memperkenalkan dua perangkat yang banyak digunakan di kalangan akademik maupun industri, yaitu MATLAB dan Python3. Kedua perangkat ini dipilih karena relevansinya yang tinggi dalam bidang keteknikan serta fleksibilitasnya dalam mendukung berbagai pendekatan pemecahan masalah. Meskipun demikian, penggunaan perangkat hanyalah sarana, esensi utama tetap terletak pada pemahaman konsep, alur logika, serta keterampilan berpikir komputasional mahasiswa.

Akhir kata, penulis menyadari bahwa modul ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan di masa mendatang. Semoga modul ini dapat memberikan manfaat yang sebesar-besarnya bagi mahasiswa dalam mengembangkan wawasan, kemampuan berpikir, dan keterampilan pemrograman dasar, sehingga siap menghadapi tantangan di dunia keteknikan maupun bidang lainnya.

Lhokseumawe, Agustus 2025

Tim Dosen Pengampu MK

Lembar Pengesahan

Nama Dokumen : Modul Ajar Logika Pemrograman
Nama Mata Kuliah : Logika Pemrograman
Kode Mata Kuliah : TMS0633
Jumlah Pertemuan : 16
Jumlah Bahan Kajian : 10
Dosen Pengampu Mata Kuliah : 1. Ahmad Nayan, S.T., M.T. (Koordinator)
2. Asnawi, S.T., M.Sc.
3. Ir. Shaki Saptiadi Putra, S.T., M.Eng.

Mengetahui,
Ketua Program Studi

Lhokseumawe, 29 Agustus 2025
Koordinator/Pengampu

(Abdul Rahman, S.T. M.Eng)
NIP. 196811202003121001

(Ahmad Nayan, S.T., M.T.)
NIP. 197504262002121001

Mengetahui,
Ketua Jurusan Teknik Mesin

(Asnawi, S.T., M.Sc.)
NIP. 198002272006041010

Daftar Isi

Prakata	ii
1 Konsep Dasar Pemrograman: Algoritma, <i>Flowchart</i>, dan Bahasa Pemrograman	1
1.1 Algoritma	1
1.1.1 Karakteristik Algoritma yang Baik	1
1.1.2 Contoh Algoritma Sederhana	2
1.2 <i>Flowchart</i>	2
1.2.1 Simbol-simbol Dasar pada <i>Flowchart</i>	2
1.3 Bahasa Pemrograman	3
1.3.1 <i>Low-Level Language vs High-Level Language</i>	3
1.3.2 MATLAB dan Python dalam Konteks Teknik Mesin	4
1.4 Contoh Studi Kasus	4
2 Struktur Data: Tipe Data, Variabel, dan Operator	6
2.1 Pendahuluan	6
2.2 Tipe Data	6
2.2.1 Klasifikasi Umum	6
2.2.2 Catatan Penting Spesifik Bahasa	7
2.2.3 Perbandingan Singkat Tipe Data Dasar	7
2.2.4 Indeks, <i>Slicing</i> , dan Mutabilitas	7
2.3 Variabel	8
2.3.1 Konsep dan Penetapan Nilai	8
2.3.2 Aturan Penamaan dan Konvensi	8
2.3.3 Lingkup (<i>Scope</i>)	8
2.4 Operator	9
2.4.1 Operator Aritmetika	9
2.4.2 Operator Perbandingan dan Logika	9
2.4.3 Operator Khusus	9

2.5	Contoh Studi Kasus: Energi Kinetik	10
2.6	Ringkasan	10
3	Percabangan Logika	11
3.1	Pendahuluan	11
3.2	Konsep Dasar Percabangan	11
3.2.1	Operator Relasional	11
3.2.2	Operator Logika	12
3.3	Struktur Percabangan	13
3.3.1	Percabangan <code>if</code>	13
3.3.2	Percabangan <code>if-else</code>	13
3.3.3	Percabangan <code>if-elseif-else</code>	14
3.3.4	Operator Logika Dasar	14
3.3.5	Percabangan Bersarang (<i>Nested If</i>)	15
3.4	Studi Kasus: Penentuan Kondisi Suhu Ruangan	15
3.5	Studi Kasus: Penentuan Status Mahasiswa	16
3.6	Ringkasan	17
4	Kalang Perulangan	18
4.1	Pendahuluan	18
4.2	Konsep Dasar Perulangan	18
4.3	Kalang <code>for</code>	18
4.3.1	Sintaks dan Implementasi	19
4.3.2	Contoh: Menghitung Faktorial	19
4.4	Kalang <code>while</code>	19
4.4.1	Sintaks dan Implementasi	19
4.4.2	Contoh: Mencari Akar Kuadrat dengan Metode Iterasi	20
4.5	Latihan Studi Kasus	20
4.6	Ringkasan	21
5	<i>Array</i> dan <i>String</i>	22
5.1	Pendahuluan	22
5.2	<i>Array</i> (Larik)	22
5.2.1	Deklarasi dan Akses Elemen	22
5.3	<i>Array</i> Multidimensi (Matriks)	23
5.3.1	Contoh: Matriks Transformasi	23
5.4	<i>String</i>	23
5.4.1	Operasi Dasar pada <i>String</i>	23

5.5	Latihan Studi Kasus	24
5.6	Ringkasan	24
6	Fungsi dan Parameter	25
6.1	Pendahuluan	25
6.2	Konsep Fungsi	25
6.3	Mendefinisikan dan Memanggil Fungsi	25
6.3.1	Contoh: Fungsi Konversi Suhu	26
6.4	Parameter dan Argumen	26
6.4.1	Parameter dengan Nilai <i>Default</i>	26
6.5	Latihan Studi Kasus	27
6.6	Ringkasan	28
7	Pemrograman Berbasis Objek (OOP)	29
7.1	Pendahuluan	29
7.2	Konsep Dasar OOP	29
7.2.1	Kelas dan Objek	29
7.2.2	Contoh: Kelas <code>MotorListrik</code>	29
7.3	Pilar Utama OOP	31
7.3.1	Enkapsulasi	31
7.3.2	Abstraksi	31
7.3.3	Inheritansi (Pewarisan)	31
7.3.4	Polimorfisme	32
7.4	Latihan Studi Kasus	32
7.5	Ringkasan	32
8	Manajemen Berkas	33
8.1	Pendahuluan	33
8.2	Membaca dan Menulis Berkas Teks	33
8.2.1	Langkah-langkah Dasar	33
8.2.2	Contoh: Menulis Data Eksperimen	33
8.2.3	Contoh: Membaca Data Eksperimen	34
8.3	Penanganan Kesalahan (<i>Error Handling</i>)	35
8.4	Latihan Studi Kasus	35
8.5	Ringkasan	35
9	Analisis dan Visualisasi Data	36
9.1	Pendahuluan	36
9.2	Analisis Data Dasar	36

9.2.1	Studi Kasus: Analisis Data Temperatur	36
9.3	Visualisasi Data	37
9.3.1	Contoh: Grafik Garis dan Diagram Batang	37
9.4	Latihan Studi Kasus	38
9.5	Ringkasan	39
10	<i>Debugging</i> dan Dokumentasi	40
10.1	Pendahuluan	40
10.2	<i>Debugging</i>	40
10.2.1	Jenis-jenis Kesalahan	40
10.2.2	Strategi <i>Debugging</i>	40
10.3	Dokumentasi Kode	41
10.3.1	Komentar dalam Kode	41
10.3.2	<i>Docstrings</i> dan Bantuan Fungsi	41
10.4	Latihan Studi Kasus	42
10.5	Ringkasan	42
	Referensi	43

Bab 1

Konsep Dasar Pemrograman: Algoritma, *Flowchart*, dan Bahasa Pemrograman

1.1 Algoritma

Algoritma merupakan inti dari pemrograman. Secara umum, algoritma didefinisikan sebagai urutan langkah-langkah yang logis, sistematis, dan terbatas untuk menyelesaikan suatu permasalahan tertentu. Kata "logis" berarti setiap langkah dapat diterima akal dan tidak ambigu. "Sistematis" artinya langkah-langkah tersusun secara berurutan dan terstruktur. "Terbatas" menunjukkan bahwa algoritma pasti berakhir setelah sejumlah langkah tertentu.

Dalam bidang keteknikan, algoritma sangat bermanfaat. Misalnya pada teknik mesin:

- Menghitung **efisiensi mesin** berdasarkan output dan input energi.
- Menganalisis **gaya** yang bekerja pada sistem mekanik.
- Menghitung **proses termodinamika**, seperti siklus Rankine atau Otto.

1.1.1 Karakteristik Algoritma yang Baik

1. **Jelas dan tidak ambigu** : setiap langkah harus dapat dipahami tanpa penafsiran ganda.
2. **Efektif** : algoritma harus dapat dijalankan secara nyata dalam waktu dan sumber daya terbatas.

3. **Terstruktur** : langkah-langkah ditulis secara urut, sehingga mudah diterjemahkan ke dalam program komputer.
4. **Generik** : dapat digunakan untuk menyelesaikan masalah yang sejenis, bukan hanya satu kasus khusus.

1.1.2 Contoh Algoritma Sederhana

Menghitung kecepatan rata-rata berdasarkan jarak dan waktu tempuh:

1. Masukkan jarak tempuh (s).
2. Masukkan waktu tempuh (t).
3. Hitung kecepatan rata-rata $v = s \div t$.
4. Tampilkan hasil kecepatan.

1.2 *Flowchart*

Flowchart adalah representasi visual dari algoritma dalam bentuk simbol-simbol standar. *Flowchart* membantu memvisualisasikan alur logika, sehingga lebih mudah dipahami, dianalisis, dan dikomunikasikan, baik oleh manusia maupun saat diterjemahkan ke dalam bahasa pemrograman.

1.2.1 Simbol-simbol Dasar pada *Flowchart*

- **Oval** : simbol mulai atau selesai.
- **Persegi panjang** : proses atau perhitungan.
- **Belah ketupat** : keputusan (ya/tidak).
- **Panah** : alur eksekusi program.

	Flow Direction symbol Yaitu simbol yang digunakan untuk menghubungkan antara simbol yang satu dengan simbol yang lain. Simbol ini disebut juga connecting line.		Simbol Manual Input Simbol untuk pemasukan data secara manual on-line keyboard
	Terminator Symbol Yaitu simbol untuk permulaan (start) atau akhir (stop) dari suatu kegiatan		Simbol Preparation Simbol untuk mempersiapkan penyimpanan yang akan digunakan sebagai tempat pengolahan di dalam storage.
	Connector Symbol Yaitu simbol untuk keluar - masuk atau penyambungan proses dalam lembar / halaman yang sama.		Simbol Predefine Proses Simbol untuk pelaksanaan suatu bagian (sub-program)/prosedure
	Connector Symbol Yaitu simbol untuk keluar - masuk atau penyambungan proses pada lembar / halaman yang berbeda.		Simbol Display Simbol yang menyatakan peralatan output yang digunakan yaitu layar, plotter, printer dan sebagainya.
	Processing Symbol Simbol yang menunjukkan pengolahan yang dilakukan oleh komputer		Simbol disk and On-line Storage Simbol yang menyatakan input yang berasal dari disk atau disimpan ke disk.
	Simbol Manual Operation Simbol yang menunjukkan pengolahan yang tidak dilakukan oleh computer		Simbol magnetik tape Unit Simbol yang menyatakan input berasal dari pita magnetik atau output disimpan ke pita magnetik.
	Simbol Decision Simbol pemilihan proses berdasarkan kondisi yang ada.		Simbol Punch Card Simbol yang menyatakan bahwa input berasal dari kartu atau output ditulis ke kartu
	Simbol Input-Output Simbol yang menyatakan proses input dan output tanpa tergantung dengan jenis peralatannya		Simbol Dokumen Simbol yang menyatakan input berasal dari dokumen dalam bentuk kertas atau output dicetak ke kertas.

Gambar 1.1: Ragam Bentuk Blok pada *Flowchart*

1.3 Bahasa Pemrograman

Setelah algoritma dan flowchart disusun, langkah selanjutnya adalah mengimplementasikannya dalam bentuk bahasa yang dapat dipahami komputer, yaitu **bahasa pemrograman**.

1.3.1 *Low-Level Language vs High-Level Language*

Bahasa pemrograman dapat dibagi menjadi dua tingkatan:

- ***Low-Level Language*** Bahasa tingkat rendah berhubungan langsung dengan mesin (*hardware*). Contohnya adalah *assembly language* atau bahasa mesin (kode biner 0 dan 1). Bahasa ini sangat cepat dieksekusi, tetapi sulit dipahami manusia.
- ***High-Level Language*** Bahasa tingkat tinggi menggunakan sintaks yang lebih dekat dengan bahasa manusia, sehingga lebih mudah dipahami dan dipelajari. Contoh: MATLAB, Python, C, dan Java. Bahasa ini memerlukan proses kompilasi atau interpretasi untuk dapat dijalankan oleh komputer.

1.3.2 MATLAB dan Python dalam Konteks Teknik Mesin

Dalam modul ini, dua bahasa tingkat tinggi akan digunakan, yaitu:

- **MATLAB** – dikembangkan khusus untuk komputasi numerik, visualisasi data, dan pemodelan matematis. Sangat populer di bidang teknik.
- **Python** – bahasa pemrograman serbaguna dengan ekosistem pustaka yang luas, termasuk untuk komputasi ilmiah (NumPy, SciPy, Matplotlib), kecerdasan buatan, hingga otomasi.

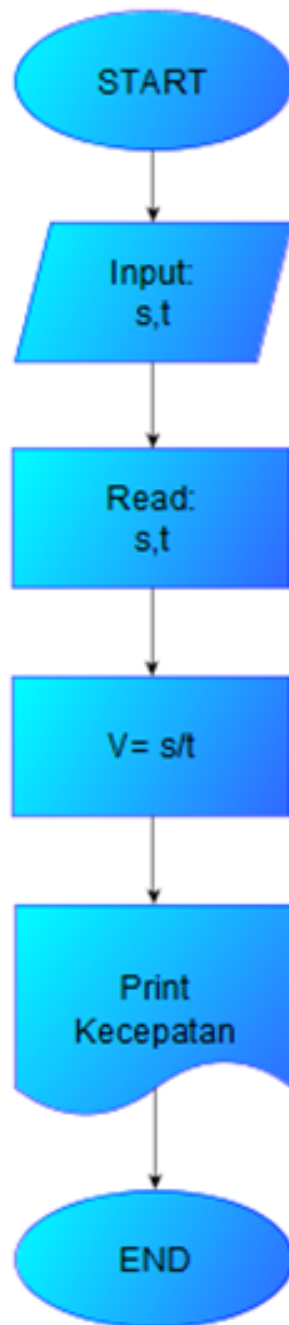
Tabel 1.1: Komparasi MATLAB dan Python3

Aspek	MATLAB	Python3
Bahasa	<i>Proprietary</i> , berbayar	<i>Open-source</i> , gratis
Fokus utama	Komputasi numerik, simulasi, analisis data	<i>General-purpose</i> , saintifik, AI, web, otomasi
Sintaks dasar	<code>a = 1:5</code>	<code>a = range(1,6)</code>
Perulangan	<pre>for i=1:5 disp(i) end</pre>	<pre>for i in range(1,6): print(i)</pre>
Plotting	<pre>x=0:0.1:10; plot(x,sin(x))</pre>	<pre>import matplotlib.pyplot as plt import numpy as np x=np.arange(0,10,0.1) plt.plot(x,np.sin(x))</pre>
Kelebihan	<i>Library</i> bawaan lengkap, mudah dipakai untuk teknik	Ekosistem luas, integrasi dengan AI, web, <i>big data</i>
Kekurangan	Mahal, kurang fleksibel di luar saintifik	Butuh library tambahan, butuh eksplorasi di awal

1.4 Contoh Studi Kasus

Berdasarkan penjelasan sebelumnya, susunlah algoritma dan flowchart untuk menghitung kecepatan rata-rata kendaraan jika jarak tempuh dan waktu diketahui. Petunjuk: gunakan rumus $v = \frac{s}{t}$.

- Input: s (jarak), t (waktu).
- Proses: $v = s \div t$.
- Output: nilai kecepatan v .



Gambar 1.2: *Flowchart* Studi Kasus: Menghitung Kecepatan Rata-rata

Bab 2

Struktur Data: Tipe Data, Variabel, dan Operator

2.1 Pendahuluan

Pada bab ini dibahas fondasi utama dalam pemrograman, yaitu **tipe data**, **variabel**, dan **operator**. Tipe data menentukan sifat nilai (misalnya bilangan, teks, logika), variabel adalah **wadah bernama** untuk menyimpan nilai, sedangkan operator adalah simbol atau kata kunci yang digunakan untuk melakukan operasi pada data. Pemahaman kuat pada tiga aspek ini akan memudahkan analisis, pemodelan, serta perhitungan numerik yang sering ditemui dalam rekayasa teknik mesin, misalnya konversi satuan, perhitungan energi, hingga pengolahan data hasil eksperimen.

2.2 Tipe Data

2.2.1 Klasifikasi Umum

Secara umum, tipe data yang sering dipakai:

- **Numerik**: bilangan bulat (integer), bilangan real (float/double), dan bilangan kompleks.
- **Boolean**: nilai logika benar/salah (`True/False` di Python; `true/false` di MATLAB).
- **Teks**: karakter dan string (teks/kalimat).
- **Koleksi/struktur**: larik (array/list), himpunan (set), kamus (dictionary), struktur `struct` (MATLAB), `cell array` (MATLAB).

2.2.2 Catatan Penting Spesifik Bahasa

- **MATLAB**: tipe numerik *default* adalah `double-precision`. Indeks array dimulai dari **1**.
- **Python 3**: `int` tidak dibatasi ukuran tetap (arbitrary precision), `float` menggunakan representasi `double-precision`. Indeks urutan **dimulai dari 0**. String adalah Unicode.

2.2.3 Perbandingan Singkat Tipe Data Dasar

Tabel 2.1: Padanan tipe data dasar dan struktur sederhana di Python 3 dan MATLAB.

Jenis	Python 3	MATLAB
Bilangan bulat	<code>x = 10 (int)</code>	<code>x = int32(10);</code> atau <code>x = 10; (disimpan double)</code>
Bilangan real	<code>y = 3.14 (float)</code>	<code>y = 3.14; (double)</code>
Bilangan kompleks	<code>z = 3+4j</code>	<code>z = 3+4i;</code>
Boolean	<code>flag = True</code>	<code>flag = true;</code>
String	<code>nama = "Mesin" (Unicode)</code>	<code>nama = "Mesin"; (string)</code> atau <code>'Mesin' (char array)</code>
Array/vektor	<code>arr = [1,2,3] (list)</code>	<code>arr = [1 2 3]; (array/matriks)</code>
Struktur	<code>d = {"rho":1000, "mu":1e-3} (dict)</code>	<code>s.rho = 1000; s.mu = 1e-3; (struct)</code>

2.2.4 Indeks, *Slicing*, dan Mutabilitas

- **MATLAB** menggunakan indeks mulai dari 1: `A(1)` adalah elemen pertama.
- **Python** menggunakan indeks mulai dari 0: `A[0]` adalah elemen pertama.
- **Mutabilitas**: di Python, `list`, `dict`, dan `set` dapat diubah; `tuple` dan `str` tidak dapat diubah. Di MATLAB, array numerik umumnya mengikuti *copy-on-write*.

Contoh (MATLAB):

```

1      A = [10 20 30 40 50];
2      pertama = A(1);           % 10
3      sub = A(2:4);             % [20 30 40]

```

Contoh (Python):

```

1      A = [10, 20, 30, 40, 50]
2      pertama = A[0]            # 10
3      sub = A[1:4]              # [20, 30, 40]

```

2.3 Variabel

2.3.1 Konsep dan Penetapan Nilai

Variabel adalah nama simbolik yang merujuk pada suatu nilai di memori. Penetapan nilai (*assignment*) mengikat nama ke nilai.

Python:

```

1      x = 10                    # x merujuk ke 10
2      x = "mesin"              # sekarang x merujuk ke string

```

MATLAB:

```

1      x = 10;                  % x bernilai 10 (double)
2      x = "mesin";             % kini x bertipe string

```

2.3.2 Aturan Penamaan dan Konvensi

- Tidak boleh diawali angka; hanya huruf, angka, dan garis bawah.
- Hindari memakai nama fungsi bawaan sebagai variabel (misalnya `sum`, `min`).
- **Python:** gunakan `snake_case` (misalnya `massa_jenis`).
- **MATLAB:** nama deskriptif singkat; konsisten dalam gaya penulisan (misalnya `massaJenis`).

2.3.3 Lingkup (*Scope*)

- **Python:** variabel *default* bersifat lokal di fungsi; `global` dan `nonlocal` tersedia.

- **MATLAB:** variabel di dalam fungsi terpisah dari *base workspace*. Kata kunci `global` ada, namun sebaiknya dihindari.

2.4 Operator

2.4.1 Operator Aritmetika

- **Python:** `+` `-` `*` `/` `//` `%` `**`
- **MATLAB:** `+` `-` `*` `/` `\` `^` `mod`

Contoh (Python):

```
1 a, b = 7, 3
2 print(a+b, a-b, a*b, a/b, a//b, a%b, a**b)
```

Contoh (MATLAB):

```
1 a = 7; b = 3;
2 disp([a+b, a-b, a*b, a/b, floor(a/b), mod(a,b), a^b])
```

2.4.2 Operator Perbandingan dan Logika

- **Python:** `==` `!=` `>` `<` `>=` `<=` serta `and` `or` `not`
- **MATLAB:** `==` `~=` `>` `<` `>=` `<=` serta `&&` `|` `|`

Contoh (Python):

```
1 x = 10; y = 5
2 print(x > y, x == y, (x>0 and y>0))
```

Contoh (MATLAB):

```
1 x = 10; y = 5;
2 disp([x>y, x==y, (x>0 && y>0)])
```

2.4.3 Operator Khusus

- **Python:** `in`, `not in`, `is`, `is not`.
- **MATLAB:** operator vektor/matriks elemen-per-elemen (misalnya `.*`, `./`, `.^`).

2.5 Contoh Studi Kasus: Energi Kinetik

Kasus: diberikan massa m (kg) dan kecepatan v (m/s), hitung energi kinetik $E_k = \frac{1}{2}mv^2$.

Python:

```
1      m = 60.0
2      v = 5.0
3      Ek = 0.5 * m * v**2
4      print(f"Energi_kinetik={Ek:.2f}Joule")
```

MATLAB:

```
1      m = 60.0;
2      v = 5.0;
3      Ek = 0.5 * m * v^2;
4      fprintf('Energi_kinetik=%.2fJoule\n', Ek);
```

Latihan Studi Kasus

1. Buatlah program yang menghitung luas lingkaran dengan input jari-jari r , gunakan konstanta π bawaan di MATLAB/Python.
2. Diberikan daftar panjang sisi balok p, l, t , hitung volume dan luas permukaan.
3. Buatlah program yang menentukan apakah suatu bilangan bulat genap atau ganjil dengan memanfaatkan operator modulus.

2.6 Ringkasan

Pada bab ini Anda telah mempelajari: (1) ragam tipe data (numerik, teks, logika, koleksi), (2) konsep variabel beserta aturan penamaan dan lingkup, serta (3) operator aritmetika, perbandingan, logika, dan khusus. Pemahaman ini menjadi landasan untuk bab berikutnya tentang **Percabangan Logika** yang memanfaatkan boolean dan operator perbandingan untuk pengambilan keputusan program.

Bab 3

Percabangan Logika

3.1 Pendahuluan

Dalam pemrograman, sering kali kita perlu membuat keputusan berdasarkan kondisi tertentu. Misalnya, apakah seorang mahasiswa lulus berdasarkan nilai ujian, atau apakah sensor mendeteksi suhu melebihi batas aman. Proses pengambilan keputusan ini disebut **percabangan logika**.

Percabangan logika memungkinkan program untuk mengeksekusi instruksi yang berbeda tergantung pada hasil evaluasi suatu kondisi (benar/salah). Dengan kata lain, percabangan memberikan "jalan bercabang" dalam alur eksekusi program.

3.2 Konsep Dasar Percabangan

Percabangan logika berlandaskan pada ekspresi boolean, yaitu ekspresi yang hasilnya hanya True/False atau 1/0. Operator logika dan relasional digunakan untuk menyusun kondisi.

3.2.1 Operator Relasional

Operator relasional digunakan untuk membandingkan dua nilai. Hasilnya selalu berupa boolean.

- Sama dengan: `==`
- Tidak sama dengan: `!=`
- Lebih besar: `>`
- Lebih kecil: `<`

- Lebih besar atau sama dengan: >=
- Lebih kecil atau sama dengan: <=

Contoh di Python:

```

1      a = 10
2      b = 7
3      print(a == b)    # False
4      print(a > b)     # True
5      print(a <= 10)   # True

```

Contoh di MATLAB:

```

1      a = 10;
2      b = 7;
3      disp(a == b);    % 0
4      disp(a > b);     % 1
5      disp(a <= 10);   % 1

```

3.2.2 Operator Logika

Operator logika digunakan untuk menggabungkan beberapa kondisi.

- AND: and (Python), && atau & (MATLAB)
- OR: or (Python), || atau | (MATLAB)
- NOT: not (Python), ~ atau ~ (MATLAB)

Contoh di Python:

```

1      x = 5
2      print(x > 0 and x < 10)    # True
3      print(x < 0 or x > 10)     # False
4      print(not (x == 5))        # False

```

Contoh di MATLAB:

```

1      x = 5;
2      disp(x > 0 && x < 10);      % 1
3      disp(x < 0 || x > 10);      % 0
4      disp(~(x == 5));            % 0

```

3.3 Struktur Percabangan

Secara umum, terdapat tiga bentuk percabangan: **if**, **if-else**, dan **if-elseif-else**.

3.3.1 Percabangan if

Struktur **if** digunakan untuk mengeksekusi suatu blok kode hanya jika kondisi bernilai benar.

Contoh di Python:

```
1     nilai = 85
2     if nilai >= 75:
3         print("Lulus")
```

Contoh di MATLAB:

```
1     nilai = 85;
2     if nilai >= 75
3         disp("Lulus")
4     end
```

3.3.2 Percabangan if-else

Struktur **if-else** digunakan ketika ada dua jalur: kondisi benar atau salah.

Contoh di Python:

```
1     nilai = 60
2     if nilai >= 75:
3         print("Lulus")
4     else:
5         print("Tidak Lulus")
```

Contoh di MATLAB:

```
1     nilai = 60;
2     if nilai >= 75
3         disp("Lulus")
4     else
5         disp("Tidak Lulus")
6     end
```

3.3.3 Percabangan if-elseif-else

Struktur ini digunakan untuk membuat lebih dari dua jalur keputusan.

Contoh di Python:

```
1      nilai = 82
2      if nilai >= 90:
3          print("A")
4      elif nilai >= 80:
5          print("B")
6      elif nilai >= 70:
7          print("C")
8      else:
9          print("D")
```

Contoh di MATLAB:

```
1      nilai = 82;
2      if nilai >= 90
3          disp("A")
4      elseif nilai >= 80
5          disp("B")
6      elseif nilai >= 70
7          disp("C")
8      else
9          disp("D")
10     end
```

3.3.4 Operator Logika Dasar

Operator logika digunakan untuk menggabungkan kondisi.

- AND: and (Python), && (MATLAB)
- OR: or (Python), || (MATLAB)
- NOT: not (Python), ~ (MATLAB)

Contoh dalam Python:

```
1      x = 15
2      if x > 10 and x < 20:
3          print("x berada di antara 10 dan 20")
```

Contoh dalam MATLAB:

```
1      x = 15;
2      if x > 10 && x < 20
3          disp('x berada di antara 10 dan 20')
4      end
```

3.3.5 Percabangan Bersarang (*Nested If*)

Percabangan bersarang terjadi ketika sebuah percabangan `if` berada di dalam percabangan `if` lainnya. Struktur ini digunakan untuk mengevaluasi kondisi yang lebih kompleks secara bertahap.

Contoh dalam Python:

```
1      x = 18
2      if x > 10:
3          if x < 20:
4              print("x berada di antara 10 dan 20")
```

Contoh dalam MATLAB:

```
1      x = 18;
2      if x > 10
3          if x < 20
4              disp('x berada di antara 10 dan 20')
5          end
6      end
```

3.4 Studi Kasus: Penentuan Kondisi Suhu Ruangan

Untuk memperjelas penerapan percabangan logika, mari kita gunakan contoh nyata: sistem sensor suhu ruangan.

Kriteria:

- Suhu $< 18^{\circ}\text{C}$ $\rightarrow$ "Terlalu Dingin"
- $18\text{--}27^{\circ}\text{C}$ $\rightarrow$ "Normal"
- Suhu $> 27^{\circ}\text{C}$ $\rightarrow$ "Terlalu Panas"

Contoh di Python:

```

1      suhu = 30
2      if suhu < 18:
3          print("Terlalu_Dingin")
4      elif suhu <= 27:
5          print("Normal")
6      else:
7          print("Terlalu_Panas")

```

Contoh di MATLAB:

```

1      suhu = 30;
2      if suhu < 18
3          disp("Terlalu Dingin")
4      elseif suhu <= 27
5          disp("Normal")
6      else
7          disp("Terlalu Panas")
8      end

```

3.5 Studi Kasus: Penentuan Status Mahasiswa

Sebagai contoh penerapan percabangan logika, misalkan sebuah sistem perlu menentukan status kelulusan mahasiswa berdasarkan nilai ujian akhir:

- Nilai ≥ 85 : Sangat Memuaskan
- Nilai ≥ 70 dan < 85 : Memuaskan
- Nilai ≥ 60 dan < 70 : Cukup
- Nilai < 60 : Tidak Lulus

Contoh dalam Python:

```

1      nilai = 72
2      if nilai >= 85:
3          print("Sangat_Memuaskan")
4      elif nilai >= 70:
5          print("Memuaskan")
6      elif nilai >= 60:
7          print("Cukup")
8      else:

```

```
9      print("Tidak Lulus")
```

Contoh dalam MATLAB:

```
1      nilai = 72;  
2  
3      if nilai >= 85  
4          disp('Sangat Memuaskan')  
5      elseif nilai >= 70  
6          disp('Memuaskan')  
7      elseif nilai >= 60  
8          disp('Cukup')  
9      else  
10         disp('Tidak Lulus')  
11     end
```

3.6 Ringkasan

Percabangan logika adalah mekanisme yang sangat penting dalam pemrograman karena menentukan alur eksekusi program berdasarkan kondisi. Operator relasional dan logika digunakan untuk menyusun kondisi, sedangkan struktur `if`, `if-else`, dan `if-elseif-else` digunakan untuk mengontrol jalannya program.

Dengan memahami percabangan logika, mahasiswa dapat membangun program yang adaptif dan dinamis, seperti sistem klasifikasi nilai, sistem kendali berbasis sensor, dan banyak aplikasi lain.

Bab 4

Kalang Perulangan

4.1 Pendahuluan

Dalam pemrograman, sering kali kita perlu mengeksekusi satu atau sekelompok instruksi secara berulang. Misalnya, untuk menghitung total penjualan dari daftar 1000 item, atau untuk menganalisis data sensor dari periode waktu tertentu. Mekanisme untuk melakukan tugas yang berulang ini disebut sebagai **kalang perulangan** (*looping*).

Perulangan memungkinkan kita untuk menulis kode yang ringkas dan efisien dengan menghindari penulisan ulang instruksi yang sama berkali-kali. Kalang perulangan adalah salah satu konsep fundamental yang paling sering digunakan dalam pengembangan algoritma.

4.2 Konsep Dasar Perulangan

Secara umum, ada dua jenis perulangan utama:

- **Perulangan Berbasis Kondisi:** Kalang terus dieksekusi selama kondisi tertentu bernilai benar (`while loop`).
- **Perulangan Berbasis Penghitung:** Kalang dieksekusi sejumlah iterasi yang sudah ditentukan (`for loop`).

4.3 Kalang for

Kalang `for` digunakan ketika jumlah perulangan sudah diketahui sejak awal.

4.3.1 Sintaks dan Implementasi

Python: Kalang `for` di Python mengiterasi melalui elemen-elemen dari sebuah sekuen (seperti daftar, tuple, string, atau range).

```
1     for item in sekuen:  
2         # blok kode yang diulang
```

MATLAB: Kalang `for` di MATLAB mengiterasi melalui sebuah vektor atau larik.

```
1     for variabel = vektor  
2         % blok kode yang diulang  
3     end
```

4.3.2 Contoh: Menghitung Faktorial

Hitung faktorial dari sebuah bilangan $n!$.

Contoh di Python:

```
1     n = 5  
2     faktorial = 1  
3     for i in range(1, n + 1):  
4         faktorial = faktorial * i  
5     print(f"Faktorial dari {n} adalah {faktorial}")
```

Contoh di MATLAB:

```
1     n = 5;  
2     faktorial = 1;  
3     for i = 1:n  
4         faktorial = faktorial * i;  
5     end  
6     fprintf('Faktorial dari %d adalah %d\n',n,faktorial);
```

4.4 Kalang while

Kalang `while` digunakan ketika jumlah perulangan tidak diketahui dan kalang akan terus berjalan selama kondisi yang diberikan bernilai benar.

4.4.1 Sintaks dan Implementasi

Python:

```

1      while kondisi:
2          # blok kode yang diulang

```

MATLAB:

```

1      while kondisi
2          % blok kode yang diulang
3      end

```

4.4.2 Contoh: Mencari Akar Kuadrat dengan Metode Iterasi

Gunakan metode iterasi Newton-Raphson untuk mencari akar kuadrat dari sebuah bilangan hingga kesalahan (*error*) di bawah ambang batas tertentu.

Contoh di Python:

```

1      n = 25
2      toleransi = 1e-6
3      tebakan = n / 2
4
5      while abs(tebakan * tebakan - n) > toleransi:
6          tebakan = 0.5 * (tebakan + n / tebakan)
7
8      print(f"Akar_kuadrat_dari_{n} adalah {tebakan:.4f}")

```

Contoh di MATLAB:

```

1      n = 25;
2      toleransi = 1e-6;
3      tebakan = n / 2;
4
5      while abs(tebakan * tebakan - n) > toleransi
6          tebakan = 0.5 * (tebakan + n / tebakan);
7      end
8      fprintf('Akar_kuadrat_dari_%.0f adalah %.4f\n', n,
            tebakan);

```

4.5 Latihan Studi Kasus

1. Buatlah program yang menjumlahkan semua bilangan ganjil dari 1 sampai 100 menggunakan kalang `for`.

2. Buatlah program yang meminta input dari pengguna hingga pengguna memasukkan nilai 0, lalu hitung rata-rata dari semua nilai yang dimasukkan (kecuali nilai 0).
3. Gunakan kalang bersarang (*nested loop*) untuk membuat tabel perkalian berukuran 10x10.

4.6 Ringkasan

Pada bab ini Anda telah mempelajari: (1) konsep kalang perulangan untuk mengeksekusi instruksi berulang, (2) perbedaan antara kalang **for** (untuk jumlah iterasi yang diketahui) dan **while** (untuk kondisi yang tidak diketahui), (3) sintaks dan contoh-contoh praktis dalam Python dan MATLAB, serta (4) penerapan perulangan dalam menyelesaikan masalah teknis dan matematis.

Bab 5

Array dan *String*

5.1 Pendahuluan

Setelah memahami tipe data dasar dan struktur kontrol, bab ini akan fokus pada dua jenis struktur data penting: *array* (larik) dan *string* (teks). Dalam rekayasa, data sering kali berbentuk koleksi, seperti deret waktu, data pengukuran, atau matriks perhitungan. *Array* dan *string* adalah fondasi untuk mengorganisasi dan memproses koleksi data tersebut.

5.2 *Array* (Larik)

Array adalah kumpulan data yang diorganisasi secara berurutan dan diakses melalui indeks. Elemen-elemen dalam *array* biasanya memiliki tipe data yang sama.

5.2.1 Deklarasi dan Akses Elemen

Python: *Array* di Python sering diimplementasikan dengan `list` atau `array` numerik dari *library* seperti NumPy. Indeks dimulai dari **0**.

```
1      # List di Python
2      nilai_sensor = [10.5, 12.1, 11.8, 13.0]
3      # Mengakses elemen pertama
4      print(nilai_sensor[0]) # Output: 10.5
```

MATLAB: *Array* di MATLAB adalah tipe data dasar. Indeks dimulai dari **1**.

```
1      % Array di MATLAB
2      nilai_sensor = [10.5, 12.1, 11.8, 13.0];
3      % Mengakses elemen pertama
4      disp(nilai_sensor(1)); % Output: 10.5000
```

5.3 *Array* Multidimensi (Matriks)

Array multidimensi digunakan untuk merepresentasikan data dalam bentuk tabel, seperti matriks. Matriks adalah struktur data yang sangat umum dalam bidang keteknikan.

5.3.1 Contoh: Matriks Transformasi

Misalkan kita memiliki matriks transformasi rotasi 2D.

Contoh di Python:

```
1      import numpy as np
2      theta = np.pi / 2 # 90 derajat
3      rotasi_matriks = np.array([[np.cos(theta), -np.sin(
4          theta)],
5          [np.sin(theta), np.cos(theta)]])
6
6      print(rotasi_matriks)
7      # Mengakses elemen baris 1, kolom 2
8      print(rotasi_matriks[0][1]) # Output: -1.0
```

Contoh di MATLAB:

```
1      theta = pi / 2; % 90 derajat
2      rotasi_matriks = [cos(theta), -sin(theta);
3      sin(theta), cos(theta)];
4      disp(rotasi_matriks);
5      % Mengakses elemen baris 1, kolom 2
6      disp(rotasi_matriks(1,2)); % Output: -1.0000
```

5.4 *String*

String adalah koleksi karakter yang digunakan untuk menyimpan teks.

5.4.1 Operasi Dasar pada *String*

Operasi umum meliputi penggabungan, pemotongan (*slicing*), dan pencarian.

Contoh di Python:

```

1      nama = "Putra"
2      jurusan = "Teknik_Mesin"
3      kalimat_lengkap = nama + "_dari_" + jurusan
4      print(kalimat_lengkap) # Output: Putra dari Teknik
                                Mesin
5
6      # Slicing: mengambil karakter ke-2 sampai ke-4
7      sub_string = jurusan[1:4]
8      print(sub_string) # Output: "ekn"

```

Contoh di MATLAB:

```

1      nama = 'Putra';
2      jurusan = 'Teknik_Mesin';
3      kalimat_lengkap = [nama '_dari_' jurusan];
4      disp(kalimat_lengkap); % Output: Putra dari Teknik
                                Mesin
5
6      % Slicing: mengambil karakter ke-2 sampai ke-4
7      sub_string = jurusan(2:4);
8      disp(sub_string); % Output: ekn

```

5.5 Latihan Studi Kasus

1. Buatlah program yang meminta 5 data suhu harian, simpan dalam sebuah array, dan hitung rata-rata serta nilai maksimumnya.
2. Diberikan matriks $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$. Buatlah program yang menjumlahkan semua elemen di diagonal utamanya.
3. Buatlah program yang membalikkan sebuah string (misalnya, "robot" menjadi "tobor").

5.6 Ringkasan

Pada bab ini Anda telah mempelajari: (1) konsep *array* sebagai koleksi data terurut, (2) perbedaan cara pengindeksan *array* antara Python (mulai dari 0) dan MATLAB (mulai dari 1), (3) penggunaan *array* multidimensi atau matriks untuk masalah teknis, dan (4) *string* sebagai *array* karakter beserta operasi dasarnya.

Bab 6

Fungsi dan Parameter

6.1 Pendahuluan

Seiring dengan semakin kompleksnya program, menulis semua kode dalam satu blok besar akan menyulitkan pembacaan, pemeliharaan, dan penggunaan ulang. Konsep **fungsi** hadir untuk mengatasi masalah ini. Fungsi adalah blok kode yang dirancang untuk melakukan tugas spesifik, yang dapat dipanggil dan digunakan kembali di berbagai bagian program.

6.2 Konsep Fungsi

Fungsi memiliki tiga tujuan utama:

- **Modularity**: Memecah program besar menjadi bagian-bagian yang lebih kecil dan mudah dikelola.
- **Reusability**: Mencegah duplikasi kode; satu fungsi dapat dipanggil berkali-kali.
- **Abstraksi**: Menyembunyikan detail implementasi, sehingga pengguna fungsi tidak perlu tahu bagaimana fungsi tersebut bekerja, cukup tahu apa yang dilakukannya.

6.3 Mendefinisikan dan Memanggil Fungsi

Setiap fungsi memiliki nama, mungkin menerima *input* (**parameter**), dan bisa mengembalikan nilai (*return value*).

6.3.1 Contoh: Fungsi Konversi Suhu

Buat fungsi untuk mengkonversi suhu dari Celsius ke Fahrenheit ($F = \frac{9}{5}C + 32$).

Contoh di Python:

```
1 def celsius_ke_fahrenheit(celsius):
2     """Mengkonversi suhu dari Celsius ke Fahrenheit."""
3     fahrenheit = (9/5) * celsius + 32
4     return fahrenheit
5
6 suhu_c = 25
7 suhu_f = celsius_ke_fahrenheit(suhu_c)
8 print(f"Suhu_{suhu_c}°C_sama_dengan_{suhu_f}°F")
```

Contoh di MATLAB:

```
1 function fahrenheit = celsius_ke_fahrenheit(celsius)
2     % Mengkonversi suhu dari Celsius ke Fahrenheit.
3     fahrenheit = (9/5) * celsius + 32;
4     end
5
6 suhu_c = 25;
7 suhu_f = celsius_ke_fahrenheit(suhu_c);
8 fprintf('Suhu_%.0f°C_sama_dengan_%.1f°F\n', suhu_c ,
        suhu_f);
```

6.4 Parameter dan Argumen

Parameter adalah variabel yang didefinisikan dalam tanda kurung saat mendefinisikan fungsi. **Argumen** adalah nilai yang diberikan saat memanggil fungsi.

6.4.1 Parameter dengan Nilai *Default*

Beberapa parameter bisa diberikan nilai bawaan.

Contoh di Python:

```
1 # Fungsi menghitung volume silinder dengan default
2 jari-jari 1
3 def hitung_volume_silinder(tinggi, jari_jari=1):
4     volume = 3.14 * (jari_jari**2) * tinggi
5     return volume
```

```

5
6     vol_1 = hitung_volume_silinder(10) # Menggunakan jari-
        jari default
7     vol_2 = hitung_volume_silinder(10, 5) # Mengganti jari-
        -jari default
8
9     print(f"Volume default = {vol_1:.2f}")
10    print(f"Volume dengan r=5 = {vol_2:.2f}")

```

Contoh di MATLAB:

```

1     % MATLAB tidak memiliki parameter default seperti
        Python.
2     % Solusi alternatif dengan validasi input
3     function volume = hitung_volume_silinder(tinggi,
        jari_jari)
4     if nargin < 2
5         jari_jari = 1;
6     end
7     volume = pi * (jari_jari^2) * tinggi;
8     end
9
10    vol_1 = hitung_volume_silinder(10);
11    vol_2 = hitung_volume_silinder(10, 5);
12
13    fprintf('Volume default = %.2f\n', vol_1);
14    fprintf('Volume dengan r=5 = %.2f\n', vol_2);

```

6.5 Latihan Studi Kasus

1. Buatlah fungsi yang menerima tiga sisi segitiga sebagai parameter dan luas sebagai *return value* menggunakan rumus Heron.
2. Kembangkan fungsi dari latihan sebelumnya agar dapat menangani kasus di mana *input* tidak membentuk segitiga yang valid (misalnya, total dua sisi lebih kecil dari sisi ketiga), lalu kembalikan nilai 0.
3. Buat fungsi yang menerima sebuah *array* numerik dan mengembalikan nilai rata-rata dari *array* tersebut.

6.6 Ringkasan

Pada bab ini Anda telah mempelajari: (1) konsep fungsi untuk memodularisasi dan menggunakan kembali kode, (2) cara mendefinisikan dan memanggil fungsi dalam Python dan MATLAB, (3) perbedaan antara parameter dan argumen, dan (4) pentingnya parameter opsional atau nilai *default* untuk membuat fungsi yang lebih fleksibel.

Bab 7

Pemrograman Berbasis Objek (OOP)

7.1 Pendahuluan

Pemrograman Berbasis Objek atau *Object-Oriented Programming* (OOP) adalah salah satu paradigma pemrograman paling dominan saat ini. OOP memodelkan masalah dengan cara yang mirip dengan dunia nyata, yaitu dengan menggunakan **objek**. Setiap objek memiliki karakteristik (*properties*) dan perilaku (*methods*). Misalnya, sebuah objek "mobil" memiliki properti seperti warna dan kecepatan, serta metode seperti "maju" dan "mundur".

7.2 Konsep Dasar OOP

7.2.1 Kelas dan Objek

Kelas (*Class*) adalah sebuah *template* atau "cetak biru" yang mendefinisikan properti dan metode dari sebuah objek.

Objek (*Object*) adalah instansiasi atau wujud nyata dari sebuah kelas. Kita dapat membuat banyak objek dari satu kelas yang sama.

7.2.2 Contoh: Kelas MotorListrik

Kita akan membuat kelas sederhana untuk merepresentasikan motor listrik.

Contoh di Python:

```
1 class MotorListrik:
2     def __init__(self, nama, daya, efisiensi):
3         self.nama = nama
4         self.daya = daya # dalam Watt
```

```

5         self.efisiensi = efisiensi # dalam persen
6
7     def hitung_daya_output(self):
8         """Menghitung daya output dalam Watt."""
9         daya_output = self.daya * (self.efisiensi /
10            100)
11         return daya_output
12
13 # Membuat objek dari kelas MotorListrik
14 motor1 = MotorListrik("Motor_A", 1000, 85)
15 motor2 = MotorListrik("Motor_B", 1500, 90)
16
17 print(f"Daya_output_Motor_A: {motor1.
    hitung_daya_output()} W")
print(f"Daya_output_Motor_B: {motor2.
    hitung_daya_output()} W")

```

Contoh di MATLAB:

```

1     % MATLAB tidak memiliki sintaks kelas yang sama dengan
2     % Python
3     % Namun, konsep serupa dapat diimplementasikan dengan
4     % struct dan fungsi
5
6     motor1.nama = 'Motor_A';
7     motor1.daya = 1000; % dalam Watt
8     motor1.efisiensi = 85; % dalam persen
9
10
11     motor2.nama = 'Motor_B';
12     motor2.daya = 1500;
13     motor2.efisiensi = 90;
14
15     % Fungsi untuk menghitung daya output
16     hitung_daya_output = @(motor) motor.daya * (motor.
17         efisiensi / 100);
18
19     fprintf('Daya_output_Motor_A: %.0f W\n',
20         hitung_daya_output(motor1));
21     fprintf('Daya_output_Motor_B: %.0f W\n',
22         hitung_daya_output(motor2));

```

7.3 Pilar Utama OOP

7.3.1 Enkapsulasi

Merupakan proses pembungkusan suatu data pada kelas dengan cara mengisolasinya agar tidak dapat diakses dari luar kelasnya. Prosedur ini memungkinkan pengguna untuk cukup menggunakan data tanpa perlu mengetahui proses detail terciptanya data tersebut.

7.3.2 Abstraksi

Merupakan proses penyederhanaan kompleksitas dengan menyembunyikan detail implementasi suatu objek, sehingga pengguna hanya perlu mengetahui fungsi atau layanan yang disediakan objek tersebut.

Konsep ini sekilas mirip dengan enkapsulasi, karena keduanya sama-sama berkaitan dengan menyembunyian. Namun, perbedaannya terletak pada tingkatannya:

- **Enkapsulasi:** Enkapsulasi berfokus pada menyembunyian data dengan cara membatasi akses langsung terhadap atribut suatu objek.
- **Abstraksi:** Abstraksi berada pada level yang lebih tinggi, yaitu menyembunyikan bagaimana suatu aksi atau operasi dilakukan oleh objek.

Dengan abstraksi, pengguna cukup memahami apa yang dapat dilakukan oleh sebuah objek, tanpa harus mengetahui bagaimana objek tersebut melakukan tindakannya. Hal ini membantu mempermudah pemodelan sistem yang kompleks dan meningkatkan keteraturan dalam desain perangkat lunak.

7.3.3 Inheritansi (Pewarisan)

Kemampuan sebuah kelas baru (*subclass*) untuk mewarisi properti dan metode dari kelas yang sudah ada (*superclass*). Ini memungkinkan penggunaan kembali kode.

Contoh di Python:

```
1      class MotorDC(MotorListrik):
2          def __init__(self, nama, daya, efisiensi, tegangan
3              ):
4              super().__init__(nama, daya, efisiensi)
5              self.tegangan = tegangan # dalam Volt
6
6          def hitung_arus(self):
```

```

7         """Menghitung arus motor DC."""
8         arus = self.daya / self.tegangan
9         return arus
10
11     motor_dc = MotorDC("Motor_DC", 500, 80, 24)
12     print(f"Arus_Motor_DC: {motor_dc.hitung_arus():.2f} A"
          )

```

7.3.4 Polimorfisme

Kemampuan objek yang berbeda untuk merespons dengan cara yang unik terhadap panggilan metode yang sama. Dalam konteks ini bayangkan sebuah objek kelas seperti **olahan ayam** yang memiliki properti fungsi berupa **cara memasak gulai**, fungsi tersebut dapat digunakan pada fungsi lain dan dimodifikasi untuk mendapatkan *output* yang berbeda. Contohnya tambahan **bumbu merah** dan variasi **durasi memasak** pada fungsi **cara memasak gulai** akan menghasilkan **rendang ayam**.

7.4 Latihan Studi Kasus

1. Buatlah kelas **Titik2D** dengan properti x dan y . Tambahkan metode untuk menghitung jarak ke titik asal $(0,0)$.
2. Kembangkan kelas **Titik2D** dari latihan sebelumnya dengan membuat kelas anak **Titik3D** yang mewarisi properti x dan y serta menambahkan properti z .
3. Buatlah dua kelas: **Lingkaran** dan **Persegi**. Kedua kelas harus memiliki metode `hitung_luas()`. Tunjukkan bagaimana polimorfisme bekerja dengan memanggil metode `hitung_luas()` pada kedua objek secara berurutan.

7.5 Ringkasan

Pada bab ini Anda telah mempelajari: (1) konsep dasar Pemrograman Berbasis Objek (OOP) yang menggunakan objek untuk memodelkan masalah, (2) perbedaan antara kelas dan objek, (3) implementasi sederhana OOP dalam Python, dan (4) empat pilar utama OOP: enkapsulasi, abstraksi, inheritance, dan polimorfisme.

Bab 8

Manajemen Berkas

8.1 Pendahuluan

Sejauh ini, program yang kita buat menyimpan data hanya selama eksekusi. Begitu program selesai, data hilang. Dalam aplikasi nyata, kita sering kali perlu menyimpan data secara permanen, misalnya untuk menyimpan hasil eksperimen, membaca input dari file konfigurasi, atau mencatat log. **Manajemen berkas** adalah proses membaca dan menulis data ke dalam file pada sistem komputer.

8.2 Membaca dan Menulis Berkas Teks

8.2.1 Langkah-langkah Dasar

Tiga langkah utama dalam manajemen berkas adalah:

1. Membuka berkas dengan mode tertentu (misalnya, baca, tulis, atau tambah).
2. Melakukan operasi baca atau tulis pada berkas yang terbuka.
3. Menutup berkas untuk menyimpan perubahan dan melepaskan sumber daya.

8.2.2 Contoh: Menulis Data Eksperimen

Misalkan kita ingin menyimpan data suhu yang diukur ke dalam sebuah berkas teks.

Contoh di Python:

```
1      # Menulis data ke file 'suhu.txt'
2      data_suhu = [25.5, 26.1, 25.9, 27.0]
3      with open("suhu.txt", "w") as file:
4          file.write("Waktu,Suhu\n") # Header
```



```

5         for i, suhu in enumerate(data_suhu):
6             file.write(f"{i+1},{suhu}\n")
7         print("Data berhasil ditulis ke suhu.txt")

```

Contoh di MATLAB:

```

1     % Menulis data ke file 'suhu.txt'
2     data_suhu = [25.5, 26.1, 25.9, 27.0];
3     fid = fopen('suhu.txt', 'w');
4     fprintf(fid, 'Waktu,Suhu\n'); % Header
5     for i = 1:length(data_suhu)
6         fprintf(fid, '%d,%.1f\n', i, data_suhu(i));
7     end
8     fclose(fid);
9     disp('Data berhasil ditulis ke suhu.txt');

```

8.2.3 Contoh: Membaca Data Eksperimen

Sekarang kita akan membaca kembali data yang telah kita simpan.

Contoh di Python:

```

1     data_baca = []
2     with open("suhu.txt", "r") as file:
3         file.readline() # Lewati baris header
4         for baris in file:
5             bagian = baris.strip().split(',')
6             if len(bagian) == 2:
7                 data_baca.append(float(bagian[1]))
8     print("Data suhu yang dibaca:", data_baca)

```

Contoh di MATLAB:

```

1     data_baca = [];
2     fid = fopen('suhu.txt', 'r');
3     fgetl(fid); % Lewati baris header
4     tline = fgetl(fid); % Baca baris pertama
5     while ischar(tline)
6         bagian = strsplit(tline, ',');
7         data_baca(end+1) = str2double(bagian{2});
8         tline = fgetl(fid);
9     end
10    fclose(fid);

```

```
11     disp('Data_suhu_yang_dibaca:');
12     disp(data_baca);
```

8.3 Penanganan Kesalahan (*Error Handling*)

Saat bekerja dengan berkas, kesalahan seperti "file tidak ditemukan" sering terjadi. Kita dapat menggunakan blok 'try-except' (Python) atau validasi 'fid' (MATLAB) untuk menangani kesalahan ini.

Contoh di Python:

```
1     try:
2         with open("file_tidak_ada.txt", "r") as file:
3             isi = file.read()
4     except FileNotFoundError:
5         print("Error: Berkas tidak ditemukan.")
```

8.4 Latihan Studi Kasus

1. Buatlah program yang membaca file teks yang berisi daftar nama dan IPK mahasiswa (misalnya, 'Budi,3.75'), lalu hitung rata-rata IPK dari semua mahasiswa tersebut.
2. Modifikasi program dari latihan sebelumnya untuk menangani file yang tidak ada tanpa menghentikan program.
3. Buatlah program yang menggabungkan konten dari dua file teks menjadi satu file teks baru.

8.5 Ringkasan

Pada bab ini Anda telah mempelajari: (1) pentingnya manajemen berkas untuk penyimpanan data permanen, (2) langkah-langkah dasar dalam membaca dan menulis file teks, (3) implementasi operasi berkas menggunakan Python dan MATLAB, serta (4) cara menangani kesalahan umum yang terkait dengan berkas.

Bab 9

Analisis dan Visualisasi Data

9.1 Pendahuluan

Dalam dunia rekayasa, data mentah sering kali sulit untuk dipahami. **Analisis data** adalah proses memeriksa, membersihkan, mengubah, dan memodelkan data untuk menemukan informasi yang berguna, menarik kesimpulan, dan mendukung pengambilan keputusan. **Visualisasi data** adalah teknik untuk merepresentasikan data dalam bentuk grafis, yang membuat pola dan tren lebih mudah dikenali.

9.2 Analisis Data Dasar

Analisis data sering dimulai dengan statistik deskriptif, seperti rata-rata, median, dan standar deviasi.

9.2.1 Studi Kasus: Analisis Data Temperatur

Misalkan kita memiliki data temperatur harian selama satu minggu.

Contoh di Python:

```
1      import numpy as np
2      data_temperatur = [28.5, 29.1, 27.8, 30.2, 29.5, 28.9,
3                          27.5]
4
5      rata_rata = np.mean(data_temperatur)
6      std_dev = np.std(data_temperatur)
7      maksimum = np.max(data_temperatur)
8
9      print(f"Rata-rata: {rata_rata:.2f}")
10     print(f"Standar Deviasi: {std_dev:.2f}")
```

```
10      print(f"Maksimum: {maksimum:.2f}")
```

Contoh di MATLAB:

```
1      data_temperatur = [28.5, 29.1, 27.8, 30.2, 29.5, 28.9,
2                          27.5];
3
4      rata_rata = mean(data_temperatur);
5      std_dev = std(data_temperatur);
6      maksimum = max(data_temperatur);
7
8      fprintf('Rata-rata: %.2f\n', rata_rata);
9      fprintf('Standar Deviasi: %.2f\n', std_dev);
10     fprintf('Maksimum: %.2f\n', maksimum);
```

9.3 Visualisasi Data

Visualisasi adalah kunci untuk memahami data. Kita akan menggunakan *library* matplotlib di Python dan fungsi plot bawaan di MATLAB.

9.3.1 Contoh: Grafik Garis dan Diagram Batang

Contoh di Python:

```
1      import matplotlib.pyplot as plt
2      data_waktu = [1, 2, 3, 4, 5, 6, 7]
3      data_temperatur = [28.5, 29.1, 27.8, 30.2, 29.5, 28.9,
4                          27.5]
5
6      # Grafik Garis
7      plt.figure(figsize=(8, 4))
8      plt.plot(data_waktu, data_temperatur, marker='o')
9      plt.title('Grafik Suhu Harian')
10     plt.xlabel('Hari')
11     plt.ylabel('Suhu (°C)')
12     plt.grid(True)
13     plt.show()
14
15     # Diagram Batang
16     plt.figure(figsize=(8, 4))
17     plt.bar(data_waktu, data_temperatur)
```

```

17 plt.title('Diagram_Batang_Suhu_Harian')
18 plt.xlabel('Hari')
19 plt.ylabel('Suhu_($^\circ$C)')
20 plt.show()

```

Contoh di MATLAB:

```

1 data_waktu = [1, 2, 3, 4, 5, 6, 7];
2 data_temperatur = [28.5, 29.1, 27.8, 30.2, 29.5, 28.9,
3 27.5];
4
5 % Grafik Garis
6 figure;
7 plot(data_waktu, data_temperatur, '-o');
8 title('Grafik_Suhu_Harian');
9 xlabel('Hari');
10 ylabel('Suhu_($^\circ$C)');
11 grid on;
12
13 % Diagram Batang
14 figure;
15 bar(data_waktu, data_temperatur);
16 title('Diagram_Batang_Suhu_Harian');
17 xlabel('Hari');
18 ylabel('Suhu_($^\circ$C)');

```

9.4 Latihan Studi Kasus

1. Buatlah program yang membaca data dari file teks (misalnya `data_tekanan.txt` yang berisi Waktu,Tekanan), lalu buatlah grafik garis yang menampilkan tekanan terhadap waktu.
2. Diberikan data hasil pengujian material, buatlah program untuk menghitung nilai rata-rata, median, dan standar deviasinya, lalu visualisasikan dalam histogram.
3. Buatlah grafik 3D dari fungsi $z = f(x, y) = x^2 + y^2$ untuk rentang x dan y tertentu.

9.5 Ringkasan

Pada bab ini Anda telah mempelajari: (1) konsep analisis data dasar untuk menemukan informasi dari data mentah, (2) pentingnya visualisasi data untuk memahami pola dan tren, (3) penggunaan *library matplotlib* di Python dan fungsi plot di MATLAB untuk membuat grafik, dan (4) penerapan analisis dan visualisasi data dalam konteks rekayasa.

Bab 10

Debugging dan Dokumentasi

10.1 Pendahuluan

Tidak ada program yang sempurna saat pertama kali ditulis. Kesalahan, atau *bug*, adalah hal yang wajar dalam proses pengembangan. *Debugging* adalah proses sistematis untuk menemukan dan memperbaiki kesalahan tersebut. Selain itu, **do-kumentasi** adalah praktik penting untuk membuat kode yang mudah dipahami, baik oleh diri sendiri maupun orang lain.

10.2 *Debugging*

10.2.1 Jenis-jenis Kesalahan

- **Kesalahan Sintaks (*Syntax Errors*)**: Kesalahan tata bahasa dalam kode (misalnya, salah ketik). Python dan MATLAB akan menampilkan pesan kesalahan yang spesifik.
- **Kesalahan Runtime (*Runtime Errors*)**: Kesalahan yang terjadi saat program sedang berjalan, seperti pembagian dengan nol atau mengakses indeks array yang tidak ada.
- **Kesalahan Logika (*Logic Errors*)**: Kode berjalan tanpa error, tetapi hasilnya salah karena alur logika yang keliru. Ini adalah jenis kesalahan yang paling sulit ditemukan.

10.2.2 Strategi *Debugging*

- **Baca Pesan Kesalahan**: Pesan kesalahan sering kali memberikan petunjuk tentang lokasi dan jenis kesalahan.

- **Gunakan Pernyataan `print`/`disp`:** Masukkan pernyataan `print` atau `disp` di berbagai titik dalam kode untuk melacak nilai variabel dan alur eksekusi.
- **Gunakan *Debugger*:** Lingkungan pengembangan terpadu (IDE) seperti Visual Studio Code atau MATLAB memiliki *debugger* bawaan yang memungkinkan Anda menjalankan kode baris demi baris, melihat nilai variabel, dan menghentikan eksekusi pada titik tertentu (*breakpoint*).

10.3 Dokumentasi Kode

Kode yang baik tidak hanya berfungsi dengan benar, tetapi juga mudah dibaca dan dipahami.

10.3.1 Komentar dalam Kode

Komentar adalah baris teks yang diabaikan oleh penerjemah, digunakan untuk menjelaskan kode.

Contoh di Python:

```
1      # Baris ini menghitung luas permukaan silinder
2      volume = 2 * 3.14 * r**2 + 2 * 3.14 * r * h
```

Contoh di MATLAB:

```
1      % Baris ini menghitung luas permukaan silinder
2      volume = 2 * pi * r^2 + 2 * pi * r * h;
```

10.3.2 *Docstrings* dan Bantuan Fungsi

Docstrings (Python) adalah string yang muncul di awal definisi modul, kelas, atau fungsi untuk memberikan dokumentasi yang lebih rinci.

Contoh di Python:

```
1      def hitung_luas_silinder(r, h):
2          """
3          Menghitung luas permukaan silinder.
4
5          Args:
6          r (float): jari-jari
7          h (float): tinggi
8          Returns:
```



```
9         float: Luas permukaan silinder
10         """
11         luas = 2 * 3.14 * r**2 + 2 * 3.14 * r * h
12         return luas
```

Di MATLAB, komentar di awal fungsi akan muncul saat pengguna mengetik "`help nama_fungsi`".

10.4 Latihan Studi Kasus

1. Diberikan sebuah kode yang memiliki kesalahan logika, temukan dan perbaiki kesalahannya dengan menggunakan pernyataan `print`.
2. Tambahkan komentar yang menjelaskan setiap bagian penting dari sebuah program yang sudah ada.
3. Tulislah dokumentasi (docstring/bantuan) untuk sebuah fungsi yang Anda buat di bab sebelumnya.

10.5 Ringkasan

Pada bab ini Anda telah mempelajari: (1) pentingnya *debugging* dan tiga jenis kesalahan umum, (2) strategi efektif untuk menemukan dan memperbaiki bug, (3) pentingnya dokumentasi kode untuk meningkatkan keterbacaan, dan (4) praktik-praktik terbaik dalam menulis komentar dan dokumentasi fungsi.

Referensi

- Hunt, Brian R., Lipsman, Ronald L., & Rosenberg, Jonathan M. (2001). *A Guide to MATLAB for Beginners and Experienced Users*. Cambridge University Press.
- Suarga. (2004). *Algoritma Pemrograman*. Penerbit Andi, Yogyakarta.
- Attaway, Stormy. (2009). *Matlab: A Practical Introduction to Programming and Problem Solving*. College of Engineering, Boston University, Boston, MA.
- Palm, William J. (2011). *Introduction to MATLAB for Engineers* (3rd ed.). McGraw-Hill, New York, NY.
- Shaw, Zed A. (2013). *Learn Python the Hard Way: A Very Simple Introduction to the Terrifyingly Beautiful World of Computers and Code*. Pearson Education, United Kingdom.
- Zarman, Wendi & Mochamad Fajar Wicaksono. (2020). *Implementasi Algoritma dalam Bahasa Python*. Informatika Bandung.
- Kurniawan, Agus. (2020). *Seri Belajar Mandiri: Pemrograman Python untuk Pemula*. Ilmu Data Publisher.
- Dokumentasi Resmi Python: <https://docs.python.org/3/>